



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

گزارش کار پروژه شماره ۷ گریز از سایه

اعضای گروه

امیرحسین محمدزاده - ۴۰۲۱۰۶۴۳۴

محمدامین کوهی - ۴۰۲۱۰۶۴۰۱

سیداحمد موسوی اول - ۴۰۲۱۰۶۶۴۸

نام آزمایشگاه

سامانه‌های نهفته

بهار ۱۴۰۵

نام استاد راهنما

دکتر محسن انصاری
جناب آقای شبرنگ

فهرست مطالب

۱	چکیده	۱
۱	مقدمه	۲
۱	معماری کلی سیستم	۳
۲	پیاده‌سازی سخت‌افزار	۴
۲	۱-۴ اجزای مدار و اتصالات	۲
۳	۵ پیاده‌سازی نرم‌افزار و منطق سرور	۳
۴	۱-۵ تولید رویه‌ای هزارتو (Procedural Generation)	۴
۴	۲-۵ هوش مصنوعی و مسیریابی (A* Pathfinding)	۴
۴	۳-۵ محدودیت دید و بهینه‌سازی داده‌ها (FOV Culling)	۴
۵	۶ کلاینت وب و رابط کاربری (Thin Client)	۵
۵	۱-۶ افکت چراغ‌قوه و دوربین تطبیقی	۵
۵	۲-۶ صدای تطبیقی (Adaptive Audio)	۵
۵	۷ گیم‌پلی، آزمایش و ارزیابی سیستم	۵
۷	۸ نتیجه‌گیری	۷
۷	۹ پیوست‌ها (ویدیو و کدهای منبع)	۷

۱ چکیده

پروژه Deadline Escape (گريز از سايه) يك سيستم تعاملی و يك بازی سبك بقا (Survival) مبتنی بر اينترنت اشيا (IoT) است كه با هدف ادغام مفاهيم سيستم‌های نهفته و برنامه‌نویسی سمت سرور توسعه یافته است. در اين پروژه، بازیکن با استفاده از يك كنترلر سخت‌افزاری سفارشی مبتنی بر ميكروكنترلر ESP32، كاراكتر خود را در يك هزارتوی تاریک و تولید شده به صورت رویه‌ای (Procedural) هدایت می‌کند. معماری اين سيستم بر پایه يك سرور مقتدر (Authoritative Server) طراحی شده است كه تمامی پردازش‌های سنگین اعم از فیزیک، تشخیص برخورد، مسیریابی هوش مصنوعی با الگوریتم A* و محاسبات پرتو افكنی (Raycasting) برای اعمال محدودیت دید زاویه‌ای را بر عهده دارد. كلاینت تحت وب نیز به عنوان يك نمایشگر هوشمند، وظیفه رندر گرافیکی و پخش صدای تطبیقی را ایفا می‌کند. در اين گزارش، به تفصیل به بررسی معماری سخت‌افزار، منطق نرم‌افزار، الگوریتم‌های پیاده‌سازی شده و نحوه ارزیابی سيستم پرداخته شده است.

۲ مقدمه

پیشرفت در زمینه سيستم‌های نهفته و اينترنت اشيا، مرزهای بین سخت‌افزار فیزیکی و محیط‌های نرم‌افزاری را كمرنگ کرده است. در اين راستا، پروژه Deadline Escape به عنوان يك چالش جامع برای تلفیق ارتباطات بی‌سیم، پردازش داده‌های حسگرها و توسعه يك موتور بازی‌سازی توزیع شده تعریف گردید. هدف اصلی اين پروژه، توسعه يك بازی شوتر با زاویه دید از بالا (Top-down Shooter) بود كه در آن از كیورد و ماوس سنتی استفاده نشود. در عوض، حرکت كاراكتر از طریق كچ كردن يك دیوایس فیزیکی (تشخیص شتاب و زاویه) و هدف‌گیری از طریق يك جوی‌استيك آنالوگ صورت می‌گیرد. اين رویکرد نیازمند طراحی يك بستر ارتباطی بلادرنگ (Real-time) با كمتري تاخیر (Latency) ممكن بود.

۳ معماری کلی سيستم

برای تضمین پایداری، جلوگیری از تقلب (Anti-cheat) و استفاده بهینه از پهنای باند، معماری سيستم بر اساس مدل Authoritative Server بنا شد. در اين معماری، هیچ‌گونه منطق فیزیکی در سمت كلاینت محاسبه نمی‌شود و جریان داده‌ها به شكل زیر است:

- **لایه سخت افزار (کنترلر):** داده‌های خام حسگرها را پردازش، فیلتر و نرمال‌سازی کرده و با نرخ بهینه ۸ هرتز به سرور ارسال می‌کند.
- **لایه سرور (مغز متفکر):** هسته Node.js وضعیت ورودی‌ها را دریافت کرده و با نرخ ۳۰ هرتز (۳۰ بار در ثانیه) فیزیک بازی را محاسبه می‌کند. سپس، خروجی نهایی (مختصات جدید) را با نرخ ۱۰ هرتز برای نمایشگر پخش (Broadcast) می‌کند.
- **لایه کلاینت (نمایشگر):** یک کلاینت سبک (Thin Client) است که فایل JSON، ارسالی از سرور را دریافت کرده، دوربین را روی بازیکن متمرکز می‌کند و محیط را رسم می‌نماید.

۴ پیاده‌سازی سخت‌افزار

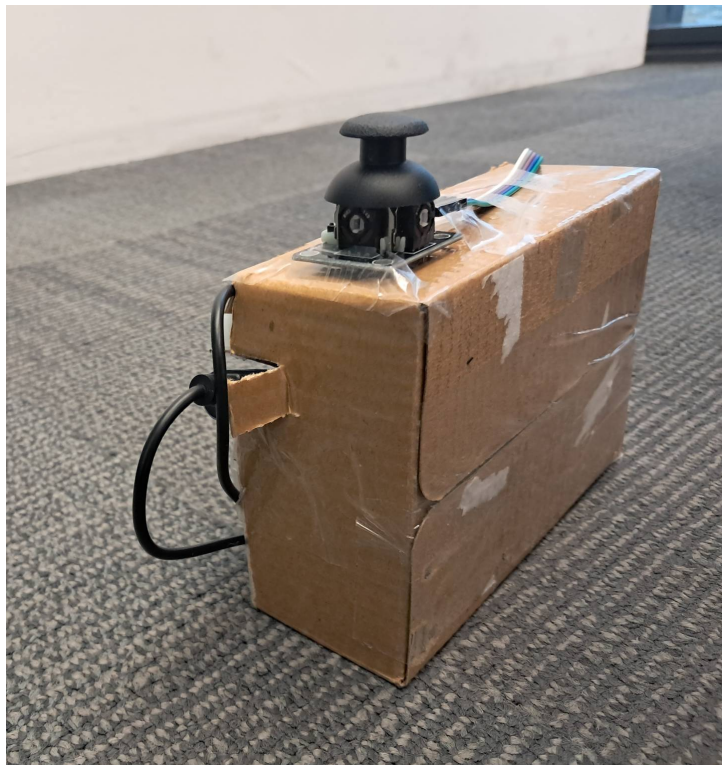
بخش سخت‌افزار بازی به عنوان نقطه اتصال فیزیکی بازیکن با محیط مجازی عمل می‌کند. این بخش روی یک بردبرد (Breadboard) مونتاژ شده و شامل قطعات زیر است:

۱-۴ اجزای مدار و اتصالات

- **میکروکنترلر ESP32:** پردازنده اصلی که از طریق شبکه Wi-Fi و پروتکل WebSocket به سرور متصل می‌شود.
 - **حسگر MPU-6050:** از طریق پروتکل I2C (پین‌های SDA=21 و SCL=22) متصل شده و زوایای Pitch و Roll را برای استخراج بردار سرعت (Velocity Vector) در محورهای X و Y محاسبه می‌کند.
 - **جوی‌استیک آنالوگ:** برای هدف‌گیری و شلیک استفاده می‌شود. خروجی‌های آنالوگ به پین‌های ADC (۳۴ و ۳۵) و دکمه شلیک به پین دیجیتال ۳۲ متصل شده است.
- به منظور افزایش دقت کنترلر، یک شعاع مرده (Radial Deadzone) برای جوی‌استیک تعریف شد تا نویزهای مقادیر آنالوگ باعث لرزش زاویه هدف‌گیری نشوند. تصاویر ۱ و ۲ جزئیات و نحوه اتصال قطعات روی بردبرد را نشان می‌دهند.



شکل ۱: نمای کلی از بردبرد، میکروکنترلر ESP32 و ماژول‌ها



شکل ۲: نمای نزدیک از جوی‌استیک آنالوگ و سنسور حرکت

۵ پیاده‌سازی نرم‌افزار و منطق سرور

هسته اصلی پروژه که با Node.js پیاده‌سازی شده، وظایف پیچیده‌ای را در پس‌زمینه انجام می‌دهد:

۱-۵ تولید رویه‌ای هزارتو (Procedural Generation)

برای اینکه ارزش تکرار بازی بالا برود، در هر بار اجرای سرور، نقشه بازی با استفاده از الگوریتم Recursive Backtracking ساخته می‌شود. این الگوریتم تضمین می‌کند که یک مسیر حل‌شدنی در شبکه ۲۰ در ۲۰ وجود داشته باشد.

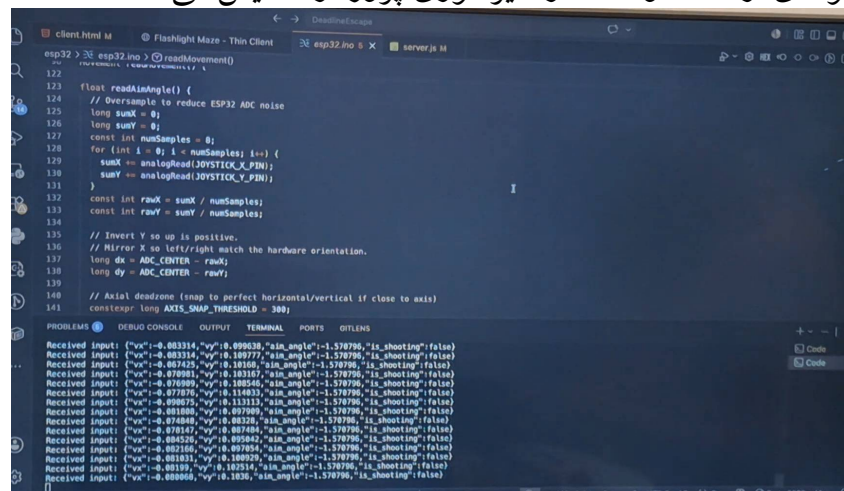
۲-۵ هوش مصنوعی و مسیریابی (A* Pathfinding)

دشمن بازی (سایه) یک هوش مصنوعی تعقیب‌کننده است. از آنجا که محیط بازی یک هزارتو است، حرکت مستقیم به سمت بازیکن باعث گیر کردن دشمن پشت دیوارها می‌شود. برای حل این مشکل، الگوریتم مسیریابی A* به صورت شبکه‌ای پیاده‌سازی شد که در فواصل زمانی مشخص، کوتاه‌ترین مسیر را به سمت بازیکن محاسبه می‌کند.

۳-۵ محدودیت دید و بهینه‌سازی داده‌ها (FOV Culling)

یکی از مهم‌ترین ویژگی‌های فنی سرور، سیستم مخفی‌سازی سایه‌ها است. سرور با استفاده از ریاضیات هندسی و الگوریتم Raycasting بررسی می‌کند که آیا دشمن در زاویه ۱۵۰ درجه‌ای روبروی بازیکن قرار دارد یا خیر. مهم‌تر از آن، چک می‌کند که آیا خط دید (Line of Sight) توسط دیوارهای هزارتو مسدود شده است یا نه. اگر دشمن در دید نباشد، مختصات آن اصلاً برای کلاینت ارسال نمی‌شود.

تصویر ۳ نمونه‌ای از کدها و ساختار دایرکتوری پروژه را نمایش می‌دهد.



```
client.html M Flashlight Maze - Thin Client esp32.ino X server.js M
esp32 > esp32.ino > readMovement()
122
123 float readAnalogAngle() {
124 // Oversample to reduce ESP32 ADC noise
125 long sumX = 0;
126 long sumY = 0;
127 const int numSamples = 8;
128 for (int i = 0; i < numSamples; i++) {
129 sumX += analogRead(JOYSTICK_X_PIN);
130 sumY += analogRead(JOYSTICK_Y_PIN);
131 }
132 const int rawX = sumX / numSamples;
133 const int rawY = sumY / numSamples;
134
135 // Invert Y so up is positive.
136 // Mirror X so left/right match the hardware orientation.
137 long dx = ADC_CENTER - rawX;
138 long dy = ADC_CENTER - rawY;
139
140 // Axial deadzone (snap to perfect horizontal/vertical if close to axis)
141 constexpr long AXIS_SNAP_THRESHOLD = 380;

PROBLEMS DEBUG CONSOLE OUTPUT TERMINAL PORTS GITLINS
Received Input: {"x":0.083314,"y":0.099638,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.083314,"y":0.109777,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.083314,"y":0.109777,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.079981,"y":0.103167,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.079981,"y":0.103167,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.077875,"y":0.114833,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.077875,"y":0.114833,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.099675,"y":0.113113,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.099675,"y":0.113113,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.074848,"y":0.08328,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.074848,"y":0.08328,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.074848,"y":0.08328,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.094526,"y":0.095842,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.094526,"y":0.095842,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.094526,"y":0.095842,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.081831,"y":0.108929,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.081831,"y":0.108929,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.081831,"y":0.108929,"aim_angle":-1.570796,"is_shooting":false}
Received Input: {"x":0.081831,"y":0.108929,"aim_angle":-1.570796,"is_shooting":false}
```

شکل ۳: ساختار فایل‌ها و بخشی از منطق Raycasting در سمت سرور

۶ کلاینت وب و رابط کاربری (Thin Client)

کلاینت بازی با استفاده از رابط برنامه‌نویسی HTML5 Canvas و بدون نیاز به هیچ موتور بازی‌سازی جانبی نوشته شده است.

۱-۶ افکت چراغ‌قوه و دوربین تطبیقی

برای ایجاد حس ترس و تعلیق، کل محیط بوم (Canvas) تاریک است. سیستم رندرینگ با استفاده از قابلیت‌های فیلتر و Clipping در جاوااسکریپت، تنها یک مخروط نوری (در جهت هدف‌گیری جوی‌استیک) را روشن می‌کند. همچنین مکانیک Follow Camera تضمین می‌کند که کاراکتر بازیکن همیشه در مرکز صفحه مرورگر ثابت بماند و این نقشه بازی باشد که جابه‌جا می‌شود.

۲-۶ صدای تطبیقی (Adaptive Audio)

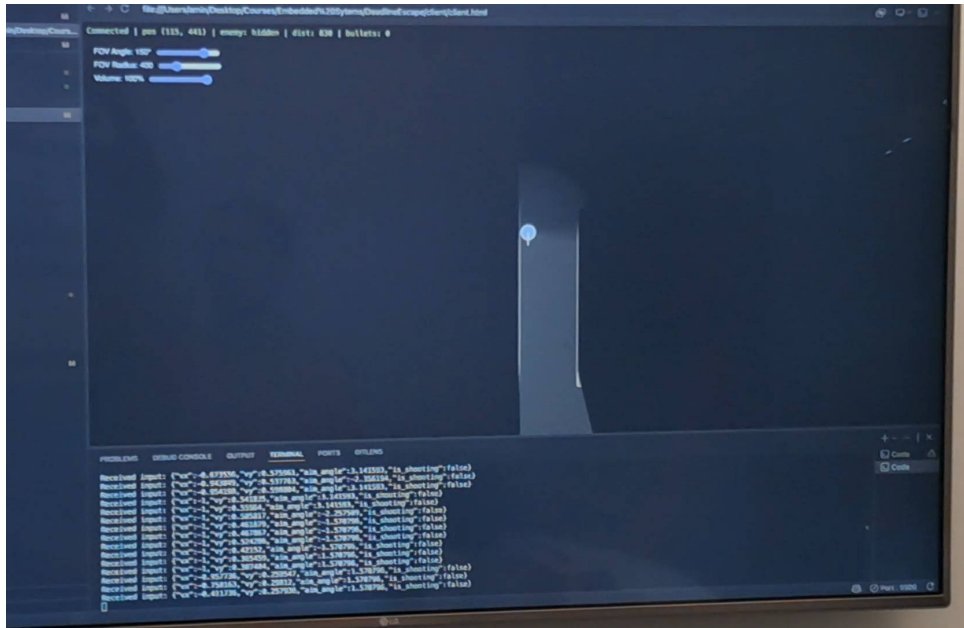
کلاینت در هر فریم فاصله هندسی (اقلیدسی) بین بازیکن و دشمن را محاسبه می‌کند. هرچه سایه به بازیکن نزدیک‌تر شود، حجم صدای قدم‌ها و تپش قلب (ولوم صوتی) به صورت لگاریتمی افزایش می‌یابد تا به بازیکن هشدار دهد.

۷ گیم‌پلی، آزمایش و ارزیابی سیستم

در مراحل تست بازی، بازیکن باید در تاریکی مطلق با استفاده از افکت چراغ‌قوه مسیر خروج را پیدا کند. اگر دشمن بیش از حد نزدیک شد، بازیکن می‌تواند با فشار دادن کلید جوی‌استیک، شلیک کرده و دشمن را برای مدت ۲ ثانیه منجمد (Stun) کند تا فرصت فرار مهیا شود.

آزمایش‌های شبکه نشان داد که حتی با تنظیم نرخ ارسال سخت‌افزار روی ۸ هرتز، بازی با درون‌یابی ذهنی بازیکن بسیار روان به نظر می‌رسد و مصرف پهنای باند به شدت کاهش می‌یابد.

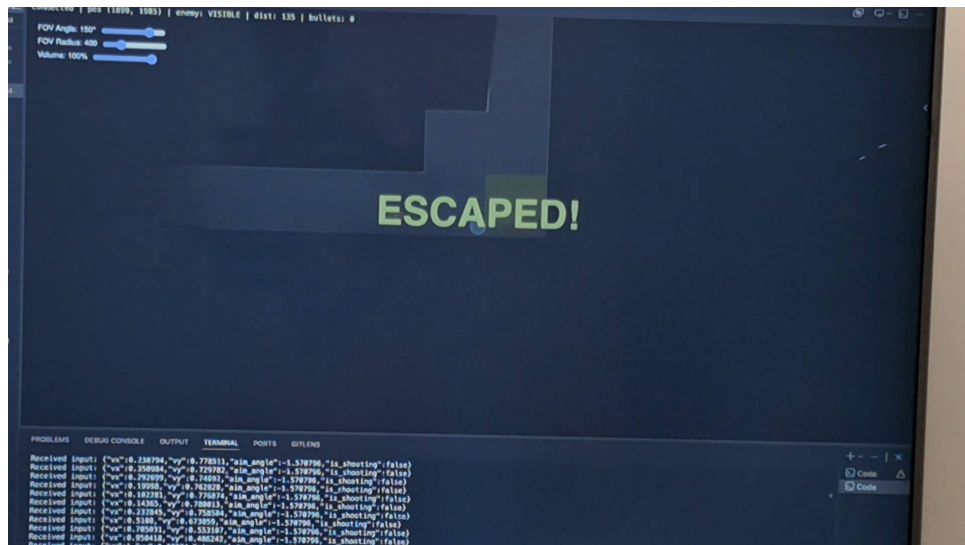
در اشکال ۴، ۵ و ۶ نماهایی از گیم‌پلی در حالات مختلف نشان داده شده است.



شکل ۴: محیط اولیه بازی و تمرکز دوربین روی کاراکتر با افکت چراغ قوه



شکل ۵: هزارتوی بازی



شکل ۶: نزدیک شدن به نقطه خروج (رنگ سبز) و مخفی ماندن دیوارهای خارج از دید

۸ نتیجه گیری

توسعه پروژه Deadline Escape تجربه‌ای جامع از چرخه کامل مهندسی یک محصول تعاملی بود. از لحیم‌کاری و راه‌اندازی سنسورها با زبان C++ گرفته تا توسعه بک‌اند بلادرنگ با Node.js و پیاده‌سازی گرافیک تحت وب. این پروژه پتانسیل بالایی برای معماری‌های Thin Client را در اینترنت اشیا اثبات کرد و نشان داد که با انتقال بار پردازشی به سرور، می‌توان حتی با سخت‌افزارهای ساده، تجربه‌های تعاملی پیچیده‌ای خلق کرد.

۹ پیوست‌ها (ویدیو و کدهای منبع)

به منظور ارزیابی دقیق‌تر و درک بهتر خروجی پروژه، یک ویدیوی جامع ضبط گردیده است. در این ویدیو، بررسی خط به خط کدها، نحوه کارکرد مدار سخت‌افزاری و همچنین اجرای زنده گیم‌پلی بازی به طور کامل نمایش داده شده و ضمیمه این گزارش گردیده است.

همچنین تمامی کدهای منبع شامل فایل‌های سرور، فرانت‌اند و کدهای آردوینو به صورت متن باز (Open Source) در مخزن گیت‌هاب زیر در دسترس می‌باشند:

<https://github.com/MohammadAminKoohi/DeadlineEscape>